



Visuel à remplacer

UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Analyse de durabilité des systèmes de propulsion par microcontrôleurs externe

Rapport de stage STXX – P20XX

NOM Prénom

Nom de la spécialité d'ingénieur
Nom de la filière

Entreprise DEMO-Contrôleurs

8 rue de la Fierté
75 013 Paris
www.democontroleurs.com

Tuteur en entreprise
NOM Prénom

Suiveur UTBM
NOM Prénom

Tuteur en entreprise
JACQUES Pierre-Paul

Introduction

Dans le cadre de ma dernière année du cursus d'ingénieur en informatique à l'Université de Technologie de Belfort-Montbéliard (UTBM), j'ai réalisé un stage de fin d'études en immersion professionnelle, ce qui constitue une étape charnière. Ce projet de fin d'études, d'une durée de six mois, m'a offert l'opportunité d'appliquer et de consolider mes compétences techniques et méthodologiques au sein d'un environnement industriel exigeant. C'est dans cette dynamique de transition vers la vie active que s'est inscrit ce stage, que j'ai effectué du 9 février au 24 juillet 2026 au sein de la société Magellium.

Fondée sur une expertise pointue dans le domaine de l'ingénierie logicielle, du traitement d'images et de la géomatique, Magellium s'impose comme un acteur de référence dans le développement de solutions technologiques innovantes. Intégré au cœur des équipes de l'agence de Ramonville-Saint-Agne, à proximité de Toulouse, j'ai effectué ce stage au sein d'un pôle technique où les problématiques de scalabilité, d'automatisation et d'architecture logicielle sont quotidiennes.

L'objectif principal de ma mission consistait à répondre à des défis technologiques d'actualité, notamment l'intégration et l'optimisation d'architectures modernes. Les travaux que j'ai menés se sont articulés autour de la mise en œuvre de solutions full-stack et DevOps, avec une ouverture prononcée vers les technologies émergentes d'orchestration d'intelligence artificielle et d'agents autonomes. Ce projet a nécessité une immersion complète dans des environnements de développement agiles, exigeant de ma part rigueur, autonomie et force de proposition.

Ce rapport s'attache à présenter de manière détaillée l'ensemble des travaux que j'ai réalisés au cours de ces six mois. Dans un premier temps, j'introduirai le contexte général de l'entreprise Magellium ainsi que son positionnement stratégique. Je détaillerai ensuite l'analyse du besoin et les choix d'architecture technique qui ont guidé mes développements. Enfin, je présenterai les phases d'implémentation, les résultats obtenus, ainsi qu'un bilan technique et personnel de cette expérience enrichissante. Cette immersion a été particulièrement fructueuse, puisqu'elle m'a permis de décrocher mon premier CDI dans le domaine de l'intelligence artificielle, confirmant ainsi la pertinence et l'impact de cette expérience professionnelle.

Remerciement

Je tiens à exprimer ma profonde gratitude à toute l'équipe de Magellium pour leur accueil bienveillant. Dès le premier jour, j'ai été intégré au sein d'une équipe dynamique et compétente, qui m'a offert un environnement de travail stimulant et propice à l'apprentissage. Je remercie particulièrement mon tuteur de stage, Mathieu Niord, pour son encadrement attentif, ses conseils avisés et sa disponibilité tout au long de ce stage. Sa guidance m'a permis de progresser rapidement et de surmonter les défis techniques rencontrés. Je suis également reconnaissant envers mes collègues pour leur soutien, leur collaboration et les échanges enrichissants qui ont jalonné cette expérience.

Cette expérience professionnelle a été très enrichissante, tant sur le plan personnel que technique. Elle m'a permis de développer de nouvelles compétences, d'approfondir mes connaissances et de me confronter à des problématiques réelles du monde industriel. Je suis convaincu que les enseignements tirés de ce stage me seront précieux pour la suite de ma carrière d'ingénieur en informatique.

Grâce à cette expérience, j'ai renforcé ma capacité d'exprimer mes idées, ma prise de décision et mon autonomie. Ces atouts, essentiels pour ma future carrière professionnelle, ont été particulièrement valorisés et m'ont permis de décrocher mon premier CDI dans le domaine du développement logiciel et de l'Intelligence Artificielle. Je suis également reconnaissant envers l'Université de Technologie de Belfort-Montbéliard (UTBM) pour m'avoir offert cette opportunité de stage, qui a été une étape cruciale dans mon parcours académique et professionnel.

Glossaire

Ce glossaire a pour but de définir les termes techniques utilisés dans ce rapport. Il est destiné à aider les lecteurs à comprendre les concepts et les technologies abordés.

- **Full-stack** : Se réfère à la capacité de travailler sur l'ensemble des couches d'une application, du front-end (interface utilisateur) au back-end (logique métier et base de données).
- **DevOps** : Un ensemble de pratiques qui combine le développement logiciel (Dev) et les opérations informatiques (Ops) pour améliorer la collaboration, l'automatisation et la livraison continue des applications.
- **CI/CD (Continuous Integration / Continuous Delivery)** : Ensemble de pratiques et de pipelines automatisés permettant de valider, tester, packager et déployer continuellement les modifications logicielles.
- **Shift-Left** : Stratégie consistant à déplacer les contrôles de qualité, de sécurité et d'exécution des tests au plus tôt dans le cycle de développement (dès le poste local du développeur).
- **Smoke Tests (Tests de fumée)** : Suite minimale de tests rapides servant de sentinelle pour vérifier qu'un composant, un service ou une route critique fonctionne correctement après déploiement ou modification.
- **Flaky Tests** : Tests instables dont le résultat alterne entre succès et échec d'une exécution à l'autre sans qu'aucune modification de code n'ait eu lieu.
- **Git Hooks (pre-commit / pre-push)** : Scripts automatisés exécutés localement par Git avant la validation d'un commit ou d'un push pour garantir le respect des normes de qualité et de formatage.
- **DTO (Data Transfer Object)** : Patron de conception (**design pattern**) servant à encapsuler et transférer des données typées entre différentes couches logicielles sans logique métier.
- **Orchestration d'intelligence artificielle** : La coordination et la gestion de différentes composantes d'une solution d'intelligence artificielle, telles que les modèles, les données et les processus, pour atteindre des objectifs spécifiques.
- **Agents autonomes** : Des systèmes informatiques capables de prendre des décisions et d'agir de manière indépendante pour accomplir des tâches spécifiques, souvent en utilisant des techniques d'intelligence artificielle.

- **Scalabilité** : La capacité d'un système à gérer une augmentation de la charge de travail ou à être facilement étendu pour accueillir plus d'utilisateurs ou de données.
- **Automatisation** : L'utilisation de technologies pour effectuer des tâches sans intervention humaine, souvent pour améliorer l'efficacité et réduire les erreurs.
- **Architecture logicielle** : La structure organisationnelle d'un système logiciel, définissant les composants, leurs interactions et les principes de conception qui guident le développement.
- **Agile** : Une méthodologie de développement logiciel qui privilégie la collaboration, la flexibilité et l'adaptation aux changements tout au long du projet.
- **PoC (Proof of Concept)** : Réalisation expérimentale à échelle réduite destinée à valider la faisabilité technique et la pertinence d'une solution ou d'une architecture.
- **RAG (Retrieval-Augmented Generation)** : Une technique d'intelligence artificielle qui combine la génération de texte avec la récupération d'informations à partir de sources externes pour produire des réponses plus précises et informées.
- **LLM (Large Language Model)** : Un modèle de langage de grande taille, souvent basé sur des architectures de réseaux de neurones, capable de comprendre et de générer du texte de manière cohérente et contextuelle.
- **Inférence** : Phase d'exécution au cours de laquelle un modèle de langage entraîné traite une invite (**prompt**) et génère une réponse.
- **Token & Fenêtre de contexte** : Unité élémentaire de découpage textuel traitée par un LLM, et volume maximal d'informations qu'un modèle peut analyser simultanément en mémoire de travail.
- **Pipelines de données** : Un ensemble de processus automatisés qui permettent de collecter, transformer et stocker des données pour les rendre utilisables dans des applications ou des analyses.
- **MCP (Model-Context-Protocol)** : Un protocole standardisé de communication entre un modèle d'intelligence artificielle et des outils mis à sa disposition, permettant de structurer les interactions et les échanges d'informations de manière sécurisée.
- **Context Engineering** : Ensemble de techniques visant à construire dynamiquement le contexte fourni à un modèle de langage plutôt qu'à lui transmettre l'intégralité des données disponibles. Il combine découpage sémantique du code, enrichissement par métadonnées, recherche hybride

(dense et lexicale) et re-ranking afin de sélectionner les informations les plus pertinentes pour une tâche donnée.

- **Embeddings (Plongements vectoriels)** : Représentations vectorielles numériques de fragments de texte capturant leur proximité sémantique dans un espace multidimensionnel.
- **Recherche hybride & BM25** : Combinaison d'une recherche vectorielle dense (sémantique) et d'une recherche lexicale exacte (algorithme BM25), essentielle pour cibler avec précision des identifiants et symboles de code.
- **Re-ranking** : Étape algorithmique intervenant après la recherche primaire pour réévaluer et réordonner les fragments récupérés selon leur pertinence vis-à-vis de la tâche cible.
- **Skeletonization (Squelettisation)** : Technique de compression de contexte consistant à masquer le corps des méthodes de classes dépendantes pour n'en conserver que les signatures et types.
- **Chain-of-Thought (CoT)** : Technique incitant le modèle de langage à expliciter les étapes de son raisonnement logique intermédiaire avant de formuler sa réponse finale.
- **Graphe d'état (State Graph)** : Modèle d'orchestration représentant le flux d'exécution sous forme de nœuds (agents/fonctions) et de transitions conditionnelles articulés autour d'un état partagé immutable.
- **Routage dynamique & Court-circuit** : Capacité d'un orchestrateur à adapter le chemin d'exécution en temps réel et à contourner des étapes superflues lorsque le contexte fourni est déjà suffisant.
- **Sorties structurées (Structured Outputs)** : Mécanisme (via Pydantic ou Instructor) contraignant le LLM à formater sa réponse selon un schéma typé et déterministe, directement validable par le code appelant.
- **Observabilité & Traçabilité LLM** : Analyse fine et enregistrement des métriques d'exécution d'un système agentique (latence, consommation de tokens, étapes intermédiaires, appels d'outils via LangFuse).
- **LangGraph** : Framework d'orchestration permettant de modéliser des systèmes multi-agents cycliques et orientés graphe d'état.
- **FastMCP** : Bibliothèque Python permettant d'implémenter rapidement des serveurs d'outils conformes à la spécification MCP.
- **LangFuse** : Plateforme open-source dédiée au traçage, à l'évaluation et au monitoring des applications basées sur des LLM.
- **Tree-sitter** : Générateur de parseurs syntaxiques incrémentaux produisant un arbre de syntaxe abstraite (AST) pour l'analyse structurelle du code source.

- **Playwright & Cypress** : Frameworks modernes d'automatisation de tests End-to-End et d'interfaces web.
- **Bruno** : Client d'API léger et scriptable facilitant l'externalisation et l'automatisation des tests de contrats d'interfaces HTTP.

Table des matières

Introduction	2
Remerciement	3
Glossaire	4
Table des matières	8
1. Présentation	11
1.1. L'entreprise Magellium	11
1.2. L'agence de Ramonville-Saint-Agne	12
2. Analyse du Fonctionnement et du Management de l'équipe	13
2.1. Structure de l'équipe	13
2.2. Organisation de l'équipe	13
2.3. Place occupée dans le service	14
2.3.1. Responsabilités et positionnement	14
2.3.2. Rôle et périmètre d'intervention	14
2.3.3. Autonomie et force de proposition technique	14
2.3.4. Standardisation et transmission des connaissances	15
2.3.5. Dynamique collaborative et rituels agiles	15
3. Travail Réalisé : Optimisation, Design Pattern et R&D Agentique	16
3.1. Axe 1 : Refonte de la stratégie de tests et optimisation CI/CD	16
3.1.1. Contexte technique, investigation et définition des objectifs ..	16
3.1.2. Les symptômes initiaux	16
3.1.3. Phase d'audit et analyse des goulots d'étranglement	17
3.1.4. Définition des objectifs	17
3.1.5. Centralisation et optimisation du contexte Spring Boot	18
3.1.6. Externalisation des contrats d'API avec Bruno	19
3.1.7. Fiabilisation Frontend	21
3.1.8. Benchmark Technique : Cypress vs Playwright	21
3.1.9. Stratégie « Shift-Left »	21
3.2. Axe Transverse : Industrialisation et Pattern SafeHydrator	22
3.2.1. Conception du GenericHydrator	22
3.2.2. Fiabilisation et Testabilité	22
3.2.3. Chronologie et Planning de la Mission	22
4. Axe 2 : R&D et implémentation d'une IA agentique autonome	24
4.1. Contexte technique, modèles locaux et cas d'usage ciblés	24
4.1.1. Contraintes liées aux données et à l'exécution locale	24
4.1.2. Cas d'usage étudiés	25
4.2. Le concept de Context Engineering	26
4.2.1. Limites de l'injection brute du contexte	26

4.2.2. Découpage sémantique du code	26
4.2.3. Enrichissement par les métadonnées	27
4.2.4. Recherche hybride	27
4.2.5. Re-ranking et skeletonization	28
4.3. Le Model Context Protocol : un catalogue d'outils métier	28
4.3.1. Du modèle de langage à l'agent	28
4.3.2. Conception du serveur FastMCP	29
4.3.3. Complémentarité entre RAG et MCP	29
4.4. Expérimentation sous Open WebUI et motivation de la migration	30
4.4.1. Une première validation fonctionnelle	30
4.4.2. Le problème de l'isolation des agents	30
4.4.3. Reformulation du problème	31
4.5. Problématique d'orchestration	31
4.6. Conception de l'orchestrateur avec LangGraph	32
4.6.1. Principe du graphe d'état	32
4.6.2. L'état partagé	32
4.6.3. Le superviseur central	33
4.6.4. Routage dynamique	34
4.6.5. Sous-graphes	34
4.7. Mise en œuvre du workflow TEST_COVERAGE	35
4.7.1. Les types de tâches :	35
4.7.2. Le Gatherer : constitution du contexte technique	36
4.7.3. L'Analyzer : identification des écarts	36
4.7.4. Le Solver : génération des tests	37
4.7.5. Pourquoi trois agents ?	37
4.8. Exécution conditionnelle et optimisation du workflow	38
4.9. Routage et boucles de rétroaction	38
4.10. Architecture orientée objet et robustesse des agents	39
4.10.1. Classe abstraite Agent	39
4.10.2. Séparation des configurations	40
4.10.3. Typage et registres	40
4.11. Sorties structurées et validation des décisions du LLM	41
4.11.1. Pydantic	41
4.11.2. Instructor et mécanisme de retry	41
4.12. Déploiement et industrialisation du prototype	42
4.12.1. Conteneurisation	42
4.12.2. Interface d'ingestion et administration	42
5. Observabilité et traçabilité	42

5.1. Déroulement d'une requête complète	43
5.2. Bilan de l'architecture	44
5.3. Limites identifiées	45
5.4. Résultats et perspectives	45
6. Synthèse et Bilan	47
6.1. Bilan technique et résultats obtenus	47
6.2. Estimation des gains pour l'entreprise	47
6.3. Bilan personnel	47
7. Conclusion	48
7.1. Synthèse globale	48
7.2. Perspectives	48
8. Annexes	49
8.1. Annexe 1 : Tableau d'Analyse Réflexive des Compétences (Obligatoire UTBM)	49
8.2. Annexe 2 : Bibliographie & Sources Techniques (Indispensable UTBM)	49
8.3. Annexe 3 : Analyse Framework de test Frontend	50
9. Contexte et Objectifs	50
10. Analyse Comparative des Frameworks de Tests E2E	50
10.1. Annexe 4 : Analyse outils de pré-commit	52
11. Analyse des Outils d'Analyse Statique Java (Pre-commit)	52
12. Sources :	53

1. Présentation

1.1. L'entreprise Magellium

Magellium, membre fondateur du *Magellium Artal Group*, est une entreprise de taille intermédiaire (ETI) spécialisée dans le développement de solutions technologiques de pointe. Ses cœurs de métier s'articulent principalement autour de la géographie numérique, du traitement d'images et de l'ingénierie logicielle avancée. Fondée en 2003 à Toulouse, Magellium a su développer une expertise unique qui la positionne aujourd'hui comme un acteur incontournable auprès des grands donneurs d'ordres des secteurs du Spatial, de la Défense, de la Sécurité, de l'Énergie et de l'Environnement.

Le groupe compte aujourd'hui plus de 250 collaborateurs répartis sur plusieurs pôles stratégiques en France, notamment à La Garenne-Colombes (région parisienne), Nantes et Ramonville-Saint-Agne (près de Toulouse). C'est au sein de cette agence toulousaine, implantée au cœur de l'écosystème aéronautique et spatial du Sud-Ouest, que s'est déroulé ce stage de fin d'études.

Magellium collabore quotidiennement avec une clientèle diversifiée et exigeante, allant des institutions publiques majeures (comme le CNES ou le CEA) aux grands groupes industriels privés et aux organismes de recherche internationaux. Cette double culture, mêlant rigueur scientifique et agilité industrielle, permet à l'entreprise de concevoir des architectures capables de répondre à des défis technologiques hautement complexes et à forte criticité.

L'offre de Magellium s'organise et se déploie autour de quatre grands domaines d'excellence :

- **Géomatique et cartographie (SIG)** : Magellium développe des solutions avancées de cartographie numérique, de gestion de données géospatiales massives et d'analyse spatiale. L'entreprise maîtrise l'ensemble de la chaîne de valeur du domaine, de l'intégration d'outils *Open Source* (comme PostGIS ou GeoServer) au déploiement de Systèmes d'Information Géographique d'envergure.
- **Traitement d'images et Observation de la Terre** : Reconnue pour son savoir-faire applicatif, l'entreprise conçoit des chaînes complexes d'analyse de flux visuels, notamment satellitaires (comme les programmes Copernicus et Sentinel). Ces algorithmes interviennent dans la surveillance environnementale, l'étude du climat ou la reconnaissance automatique de cibles en milieu contraint.

- **Ingénierie logicielle et Cloud** : Pour soutenir ces applications lourdes, les équipes de Magellium conçoivent des architectures logicielles sur mesure, scalables et résilientes. L'accent est mis sur l'adoption des pratiques DevOps, l'automatisation des pipelines d'intégration continue (CI/CD) et la conteneurisation des services.
- **Intelligence Artificielle et Agents Autonomes** : En réponse aux évolutions majeures du secteur, l'entreprise intègre des capacités d'intelligence artificielle, de machine learning et de deep learning. Ces briques permettent d'automatiser l'analyse prédictive, la classification d'images à la volée, et s'ouvrent activement sur l'orchestration de systèmes IA et d'agents autonomes.

1.2. L'agence de Ramonville-Saint-Agne

L'agence de Ramonville-Saint-Agne regroupe 200 et 249 collaborateurs et constitue le principal site toulousain de Magellium, aux côtés des implantations de La Garenne-Colombes et de Nantes. Elle héberge notamment les équipes travaillant sur des projets pour Airbus Defense and Space, dans le domaine de l'observation de la Terre et des applications agricoles, ainsi que des activités de développement logiciel et d'intégration d'intelligence artificielle.

C'est au sein de cette agence, et plus précisément de l'équipe **CNAPPS** pour le projet **Farmstar**, que s'est déroulé ce stage, portant sur l'optimisation logiciel et la recherche de solution innovante en matière de processus de développement.

2. Analyse du Fonctionnement et du Management de l'équipe

2.1. Structure de l'équipe

Le département CNAPPS (Cloud, Network, Architecture, Performance, Security & Software) de Magellium est organisé en plusieurs pôles d'expertise, chacun dédié à un domaine technologique spécifique. Ces pôles sont structurés de manière à favoriser la collaboration interdisciplinaire tout en permettant une spécialisation approfondie dans les domaines clés de l'entreprise. Parmi ces pôles, on trouve notamment l'expertise en ingénierie logicielle, le pôle infrastructure et Cloud, ainsi que les unités dédiées à l'intelligence artificielle.

2.2. Organisation de l'équipe

L'équipe au sein de laquelle j'ai évolué est structurée de manière à favoriser la collaboration et l'efficacité opérationnelle. Elle est composée de plusieurs membres aux compétences complémentaires, répartis selon les rôles suivants :

- **Chef de projet** : Responsable de la coordination globale, de la planification des tâches et de la communication avec les parties prenantes. Il joue un rôle clé dans la gestion des ressources et la prise de décision stratégique.
- **Développeurs** : Chargés de la conception, du développement et de la maintenance des solutions logicielles. Ils travaillent en étroite collaboration pour assurer la cohérence et la qualité du code produit.

Afin de coordonner les efforts et de garantir une progression fluide, des réunions quotidiennes (Daily Scrum) sont organisées. Ces échanges permettent de synchroniser l'avancement des tâches, d'identifier les obstacles et de réaligner les objectifs. Par ailleurs, des outils de gestion de projet comme Jira sont utilisés pour le suivi des tickets, l'assignation des responsabilités et la transparence de l'information.

La gestion du versionnage est assurée par Git, structuré autour de branches dédiées par fonctionnalité. Cette approche permet de maintenir un historique clair et de faciliter les revues de code, lesquelles sont systématiques pour garantir la qualité et favoriser le partage des connaissances.

2.3. Place occupée dans le service

2.3.1. Responsabilités et positionnement

En tant qu'étudiant ingénieur en dernière année, ma mission au sein du projet Farmstar s'est articulée autour de deux axes majeurs : l'optimisation des performances de l'infrastructure existante et la conception R&D d'un système d'intelligence artificielle autonome.

Ce double positionnement m'a conféré des responsabilités techniques importantes. Je rendais compte de mes avancées à mon tuteur technique (Mathieu NIORD), avec qui je validais les choix architecturaux critiques. Mon rôle consistait non seulement à implémenter des solutions, mais également à auditer le code existant, identifier les goulots d'étranglement (tels que les temps de build CI) et proposer des refontes en vue d'améliorer les performances. Les propositions que je développais devaient ainsi être validées pour leur faisabilité, leur sécurité et leur adéquation avec les standards de Magellium.

2.3.2. Rôle et périmètre d'intervention

Mon périmètre d'action s'est étendu sur l'ensemble du cycle de vie logiciel, allant du backend jusqu'à l'orchestration de modèles de langage :

- **DevOps et Qualité logicielle** : J'ai mené des audits de performance sur les pipelines d'intégration continue (CI/CD) sous GitLab, intégrant des hooks de pré-commit et pré-push (SonarLint, Checkstyle, Spotless) pour garantir la robustesse du code.
- **Ingénierie Backend et Tests** : J'ai travaillé sur la refactorisation de composants en Java (via de la réflexion) et mené la migration complète des tests d'API vers de nouveaux standards (Bruno pour l'intégration, Playwright pour l'E2E frontend).
- **R&D en IA Agentique** : J'ai conçu de bout en bout une infrastructure locale basée sur LangGraph et le Model Context Protocol (MCP). J'ai configuré des workflows multi-agents capables d'analyser du code source, de générer des plans de tests et de rédiger des tests de manière autonome.

2.3.3. Autonomie et force de proposition technique

L'une des caractéristiques principales de ce stage a été la grande autonomie qui m'a été accordée dans mes recherches. Cette liberté s'est traduite par des prises de décision impactantes :

- **Benchmark et migration d'outils** : Face aux lenteurs des tests End-to-End côté frontend, j'ai initié des benchmarks de divers framework de tests frontend. Ce choix a permis de diviser le temps d'exécution par trois (de 5m28s à 1m45s) et d'améliorer la stabilité de la suite de tests.
- **Choix architecturaux en IA** : Lors du développement de l'assistant IA, j'ai identifié les limites d'isolation des agents sous Open WebUI. J'ai alors proposé et implémenté une migration vers LangGraph, introduisant un superviseur centralisé et un routage dynamique orienté objet (POO) pour permettre une communication fluide entre agents.
- **Optimisation des ressources** : Mes initiatives sur le profilage du cache Spring Boot ont permis de réduire le temps de build local de 15 minutes à seulement 5 minutes, améliorant ainsi considérablement la vélocité des pipelines de l'équipe.

2.3.4. Standardisation et transmission des connaissances

Ne me limitant pas à l'écriture de code, j'ai pris le rôle de facilitateur technique. J'ai rédigé des guides de bonnes pratiques (au format Typst et Markdown) et enrichi le wiki du projet sur les standards d'implémentation des tests et l'utilisation des outils d'analyse statique.

La restitution a été une composante essentielle de ma mission. J'ai conçu et animé plusieurs présentations techniques formelles (notamment 28 slides sur la transformation de l'IA vers l'agentique et une restitution sur l'optimisation de la CI et de la rédaction de tests), permettant d'évangéliser ces nouvelles pratiques architecturales auprès de l'équipe.

2.3.5. Dynamique collaborative et rituels agiles

La réussite de ces chantiers repose sur une communication transparente :

- Des points de synchronisation réguliers pour remonter les blocages techniques et valider les Proof of Concept (PoC).
- Une démarche d'intégration continue stricte via GitLab, où chaque évolution faisait l'objet de revues de code.
- Une approche itérative orientée « Fast-Fail », permettant d'ajuster rapidement l'architecture logicielle en fonction des contraintes de sécurité et des retours terrains.

3. Travail Réalisé : Optimisation, Design Pattern et R&D Agentique

3.1. Axe 1 : Refonte de la stratégie de tests et optimisation CI/CD

3.1.1. Contexte technique, investigation et définition des objectifs

À mon arrivée au sein des équipes travaillant sur le projet Farmstar, le cycle de développement logiciel était ralenti par une infrastructure de tests manquant de standardisation et d'optimisation structurelle. Bien que la base de code soit fonctionnelle et réponde aux exigences métiers, la validation de nouvelles fonctionnalités souffrait de goulots d'étranglement techniques affectant directement la productivité et la boucle de feedback des développeurs.

3.1.2. Les symptômes initiaux

Lors de mes premières semaines de prise en main du projet, trois problématiques majeures se sont dégagées :

- **Lenteur prononcée de l'Intégration Continue (CI) :** Le temps d'exécution des tests du module principal s'avérait particulièrement long, s'étalant de 17 à 20 minutes en local, et atteignant jusqu'à 40 minutes sur les pipelines GitLab CI.
- **Complexité et effet « boîte noire » :** Les tests d'intégration backend étaient utilisés de manière indifférenciée, à la fois pour valider la logique métier complexe et pour vérifier de simples contrats d'API (tests End-to-End). Cette confusion des périmètres rendait la suite de tests verbeuse, et il était difficile de distinguer les tests réellement critiques des tests de « surface ».
- **Fragilité de la suite de tests du backend :** Il n'existait pas de stratégie de tests de fumée (**smoke tests**) pour les routes backend empruntés par le frontend. Ainsi, une modification du backend pouvait passer inaperçue si elle n'était pas couverte par un test d'intégration, exposant le projet à des risques de régressions non détectées lors des mises en production.
- **Fragilité de l'environnement frontend :** Les tests de fumée (**smoke tests**) permettant de valider l'état de des routes utilisés côté frontend après déploiement étaient inexistantes, exposant le projet à des risques de régressions non détectées lors des mises en production.

3.1.3. Phase d'audit et analyse des goulots d'étranglement

Pour comprendre l'origine exacte de ces lenteurs, notamment concernant la partie backend, une phase d'investigation technique a été initiée. Il était nécessaire de dépasser le simple constat de lenteur pour diagnostiquer le comportement de la JVM durant l'exécution des suites de tests.

J'ai procédé au déploiement d'outils d'audit de performance de l'application, et plus particulièrement de l'outil de métrologie « spring-test-profiler ». L'analyse attentive des rapports générés a mis en lumière une anomalie architecturale : un manquement dans la gestion du cache Spring.

Les métriques ont révélé que chaque classe de test provoquait la destruction et le redémarrage complet du contexte applicatif. Au lieu de réutiliser un environnement partagé, la JVM était forcée de recharger un monolithe de plus de 2000 beans à chaque itération. Cette « fuite de contexte » imposait une pénalité systématique d'environ 45 secondes par démarrage. Il est alors apparu évident que la source principale de l'effondrement des performances de la CI n'était pas liée à la complexité des tests eux-mêmes, mais bien à la charge d'initialisation répétée du framework.

3.1.4. Définition des objectifs

Suite à ce diagnostic, les objectifs de cette première phase de mission ont été définis afin de restructurer la stratégie de test en profondeur. L'enjeu était de résoudre les anomalies identifiées sans altérer la couverture fonctionnelle du code (éviter les régressions).

Les axes d'amélioration ont été fixés ainsi :

1. Résolution des anomalies de performance

- Identifier et éliminer les configurations provoquant la fuite de contexte Spring mise en évidence lors de l'audit.
- Centraliser le chargement de l'environnement applicatif pour garantir un **Cache Hit Rate**¹ maximal et réduire le temps d'exécution global à quelques minutes.

2. Clarification de l'architecture de test

- Séparer strictement les responsabilités : isoler la validation de la logique métier interne de la validation des points de terminaison HTTP.

¹Le **Cache Hit Rate** représente le pourcentage de requêtes où les données ont été trouvées dans le cache, évitant ainsi un accès plus lent à la source originale. Un taux élevé indique une meilleure performance.

- Déporter la vérification des contrats d'API vers un outillage externe dédié, léger et exempt du coût de démarrage du backend. (Tests d'intégration vs Tests End-to-End vs Smoke Tests)
- Combler le déficit de couverture frontend en déployant une infrastructure de tests de non-régression rapide et isolée des problématiques réseau. (Smoke Tests Frontend)

3. Sécurisation « Shift-Left »² et Standardisation

- Déplacer la vérification de la qualité et de la sécurité du code au plus près du développeur, en interceptant les anomalies avant même le déclenchement de la CI. (Pré-commit et pré-push Git Hooks)
- Rédiger un référentiel technique documentant ces nouvelles normes afin de standardiser l'écriture des tests au sein de l'équipe et de prévenir l'accumulation future de dette technique. (Guide de bonnes pratiques, Wiki interne)

Pour répondre aux objectifs fixés, la refonte s'est articulée autour de trois chantiers techniques majeurs, allant du cœur du backend jusqu'à la chaîne d'intégration continue.

3.1.5. Centralisation et optimisation du contexte Spring Boot

Le premier défi consistait à endiguer la « fuite de contexte » identifiée lors de l'audit. J'ai conçu et mis en place une classe mère globale, « AbstractIntegrationTest », dont héritent désormais toutes les classes de test. Cette architecture a permis de mutualiser les configurations complexes (« MockBean », « SpyBean ») et d'éliminer les annotations destructrices de cache, telles que « DirtiesContext ».

Parallèlement, la gestion de la sécurité asynchrone levait fréquemment des exceptions (« AccessDeniedException »). J'ai développé un utilitaire centralisé (« SecurityTestUtils ») pour forcer l'injection de contextes de sécurité fiables et liés à la base de données utilisée.

Résultat : Le temps de build local est passé de 17 minutes à 3 minutes, avec un taux d'utilisation du cache (**Cache Hit Rate**) stabilisé à 99,5%. Sur la CI GitLab, le temps d'exécution a été réduit de 70%, passant de 19:18 à 5:45.

²Le « Shift-Left » est une approche de développement logiciel qui consiste à déplacer les activités de test et de qualité vers les premières étapes du cycle de développement, afin de détecter et de corriger les défauts plus tôt et à moindre coût.

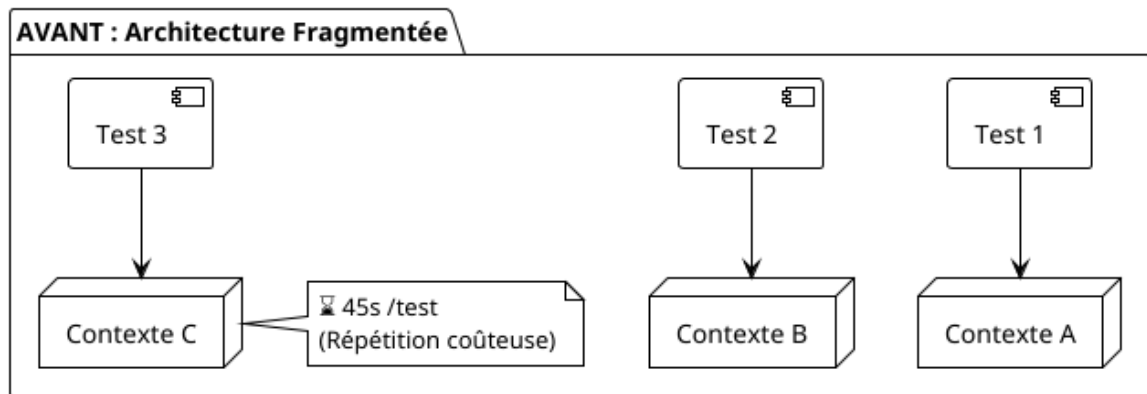


Fig. 1. – Avant Optimisation : Chaque classe de test réinitialise le contexte Spring Boot, entraînant des pénalités de performance.

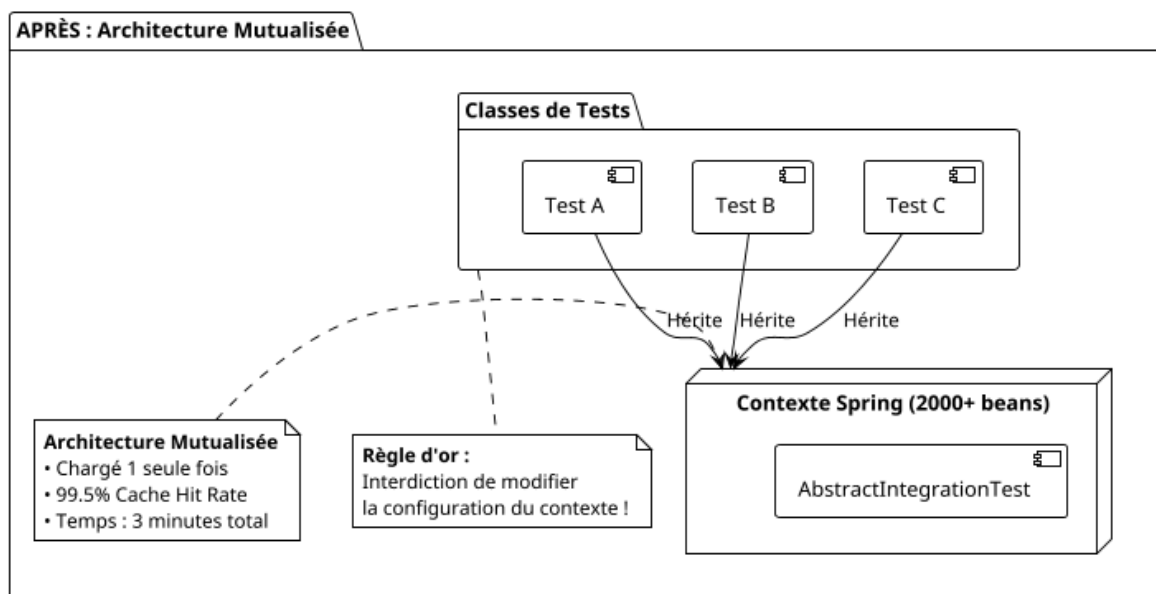


Fig. 2. – Après Optimisation : Mutualisation du contexte Spring Boot pour tous les tests d'intégration.

3.1.6. Externalisation des contrats d'API avec Bruno

Pour désengorger les tests d'intégration Spring, j'ai migré des tests de validation HTTP (« MockMvc ») vers le client d'API Bruno. Afin de rendre ces tests de bout en bout autonomes et rejouables sans polluer la base de données de la CI, une génération d'IDs dynamiques via des scripts JavaScript pré-requêtes est utilisée (génération de timestamps et UUID). Enfin, j'ai mis à jour le script Bash d'exécution des tests bruno sur la CI, afin qu'il run cette nouvelle suite de tests.

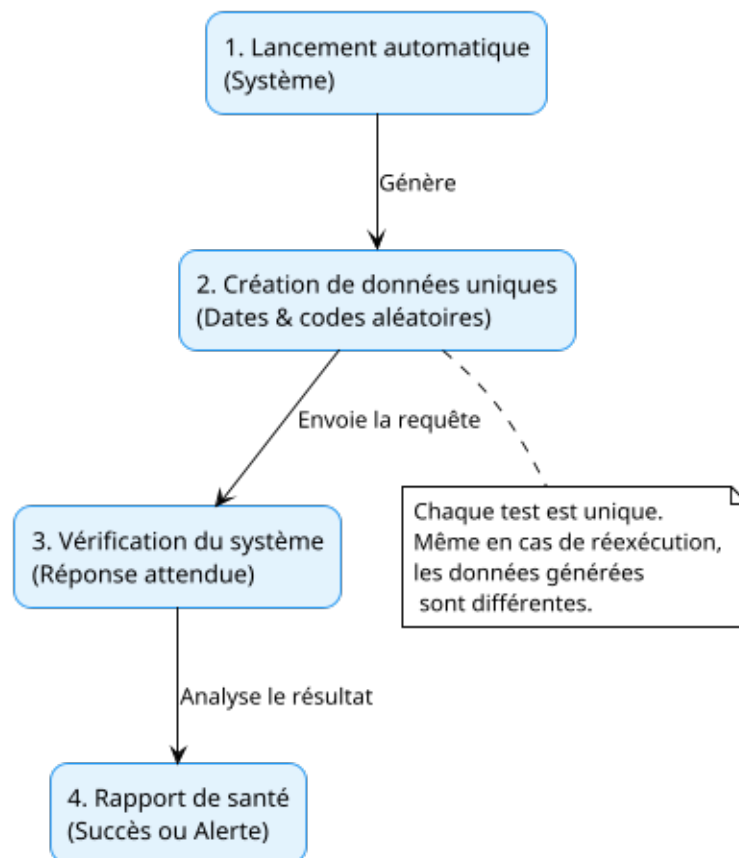


Fig. 3. – Workflow de validation des contrats d’API via Bruno

J’ai créé une suite de smoke tests couvrant l’intégralité des routes API backend sollicitées par le frontend lors de son chargement.

Objectif : Vérifier que chaque point d’entrée métier (Identité, Agronomie, Gestion de parcelles) répond aux attentes contractuelles du frontend. Ces tests agissent comme une sentinelle : ils permettent de détecter immédiatement si une modification backend « casse » la communication avec l’interface.

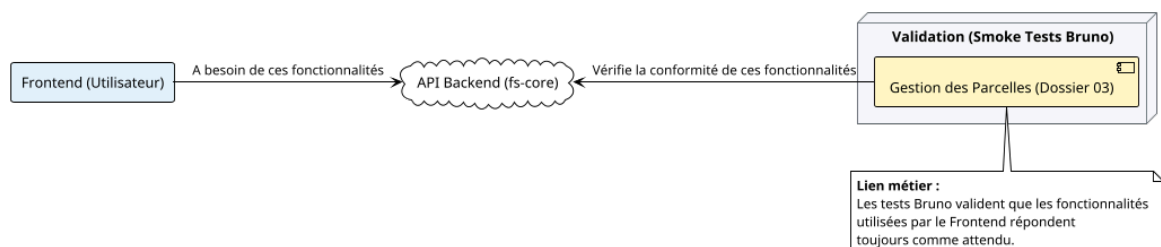


Fig. 4. – Quand est-ce qu’il s’agit d’un smoke test ?

3.1.7. Fiabilisation Frontend

Côté frontend, j'ai mis en place une suite de tests de fumée (**Smoke Tests**) pour valider les routes critiques utilisées par l'interface utilisateur. Afin de choisir quel framework de tests automatisés utiliser, j'ai mené un benchmark technique comparant Cypress et Playwright (**en annexe**). Les résultats ont montré que l'infrastructure existante sous Cypress serait trop lourde pour les besoins du projet. Une fois le choix de Playwright validé, j'ai réécrit les tests de fumée pour qu'ils soient plus rapides et plus fiables. En cours de développement, il s'est avéré que la dépendance de ces tests au lancement du backend ralentissait considérablement le cycle de la CI. Ainsi, j'ai mis en place un mock du backend pour isoler les tests frontend et réduire le temps d'exécution. Cette approche a permis de diviser le temps d'exécution des Smoke Tests par trois (de 5m 28s à 1m 45s).

3.1.8. Benchmark Technique : Cypress vs Playwright

Pour valider l'architecture de tests de fumée frontend du frontend, une étude comparative (**en annexe**) a été menée sur une suite représentative de 18 parcours utilisateurs critiques (Identity, Agronomy, Parcel Management, Monitoring).

L'avantage de Playwright réside dans sa capacité à intercepter nativement le réseau via une fonction dédiée (« injectAuthHeaders »), injectant de manière dynamique les en-têtes d'authentification (« x-authenticated-user », « x-authenticated-roles ») sans dépendre d'un sous-système complexe. De plus, le découplage des tests frontend vis-à-vis du backend via un système de moquage par fichiers « .har » a permis de sécuriser le build sur la CI GitLab.

3.1.9. Stratégie « Shift-Left »

Pour garantir la pérennité de ces optimisations, j'ai mis en place une stratégie « Shift-Left » via des hooks Git. J'ai configuré des scripts « pre-commit » (exécutant Spotless, Checkstyle et SonarLint uniquement sur les fichiers modifiés) et « pre-push » (lançant les Smoke Tests impactés). Cela permet d'intercepter les régressions directement sur le poste du développeur, avant de solliciter la CI.

3.2. Axe Transverse : Industrialisation et Pattern SafeHydrator

En parallèle des chantiers de CI et d'IA, un travail d'architecture logicielle a été mené sur le backend Java (fs-core) pour simplifier les services métier complexes (tels que StrategyService).

3.2.1. Conception du GenericHydrator

La création et la mise à jour d'entités JPA complexes nécessitaient une vérification répétitive des DTOs. J'ai développé un composant générique GenericHydrator s'appuyant sur la réflexion Java :

- @HydrationField : Définie sur les DTOs pour automatiser le mapping vers les entités.
- @HydrationGuard : Placé sur les méthodes de validation métier (ex: vérification des doublons de noms, règles d'intervalles de dates).

3.2.2. Fiabilisation et Testabilité

Pour valider ce composant sans dupliquer de code de test, j'ai mis en place une classe abstraite de test (AbstractGenericHydratorTest) exploitant la puissance des tests paramétrés JUnit 5 (@ParameterizedTest, @MethodSource). Cette approche a permis de couvrir l'intégralité des cas aux limites (**edge cases**) et d'assurer une remontée propre des exceptions métier (converties en erreurs HTTP 400).

3.2.3. Chronologie et Planning de la Mission

Période	Planning Prévisionnel	Réalisation Réelle
Février 2026	Prise en main fs-core & audit des builds	Refactoring AbstractIntegrationTest & profilage Spring
Mars 2026	Migration des tests API & CI	Portage Bruno, Playwright Frontend & Hooks Git
Avril 2026	R&D IA & FastMCP	Setup Open WebUI, FastMCP & Pattern SafeHydrator
Mai 2026	Orchestration LangGraph	Migration Multi-agents POO & Subgraphs

Juin - Juillet 2026	Industrialisation PoC & Rédaction	Module Ingestion générique, Dockerisation & Rapport ST50
---------------------	--------------------------------------	--

4. Axe 2 : R&D et implémentation d'une IA agentique autonome

La seconde partie majeure de mon stage a porté sur l'étude et l'implémentation d'un système d'intelligence artificielle agentique destiné à assister les développeurs du projet Farmstar. Contrairement aux travaux présentés dans l'axe précédent, qui concernaient principalement la fiabilisation de l'infrastructure logicielle existante, cette partie du stage s'inscrivait dans une démarche de recherche et développement.

L'objectif n'était pas simplement d'intégrer un modèle de langage dans l'environnement de développement, mais d'étudier dans quelle mesure un système capable de raisonner sur le code source, d'utiliser des outils métier et de coordonner plusieurs agents spécialisés pouvait apporter une assistance pertinente aux développeurs.

Cette problématique soulevait plusieurs difficultés. Le premier enjeu était celui de la confidentialité du code source : les données manipulées par l'agent concernent directement le patrimoine logiciel de l'entreprise et ne pouvaient donc pas être transmises à des services d'intelligence artificielle externes. Le second enjeu concernait la quantité d'information disponible. Le monorepo Farmstar contient une quantité de code largement supérieure à ce qu'un modèle de langage peut raisonnablement traiter dans un seul contexte. (code source de + 3 Millions de tokens, documentation interne, conventions de codage, etc.) Enfin, certaines tâches ne peuvent pas être réalisées à partir des seules connaissances du modèle : elles nécessitent d'explorer le dépôt, de rechercher des fichiers, de lire du code ou encore de consulter des conventions internes.

L'architecture finalement retenue repose ainsi sur trois briques complémentaires : un modèle de langage exécuté localement, un mécanisme de construction dynamique du contexte et un catalogue d'outils accessibles à l'agent. L'orchestration de ces différents composants a ensuite constitué l'objet principal du travail réalisé avec LangGraph.

4.1. Contexte technique, modèles locaux et cas d'usage ciblés

4.1.1. Contraintes liées aux données et à l'exécution locale

La première contrainte identifiée concernait la confidentialité du code de l'entreprise. Une utilisation directe d'API cloud propriétaires aurait impliqué

de transmettre à un service externe des portions potentiellement sensibles du code source et de la documentation interne.

Pour cette raison, l'expérimentation s'est orientée vers l'utilisation de modèles open-source exécutés localement. Le moteur d'inférence retenu est Ollama, permettant de déployer les modèles sur l'infrastructure disponible sans dépendre d'une API externe.

Le choix étudié s'est porté sur la famille Gemma 4 ainsi que QWen, avec plusieurs tailles de modèles correspondant à des profils d'utilisation différents. Les modèles de petite taille sont adaptés aux usages interactifs nécessitant une faible latence, tandis que les modèles disposant d'un nombre plus important de paramètres sont davantage adaptés aux tâches de raisonnement et d'orchestration plus complexes.

Le choix d'un modèle local ne résout cependant pas à lui seul la problématique. Même lorsqu'un modèle dispose d'une fenêtre de contexte importante, il reste impossible de lui transmettre systématiquement l'intégralité du monorepo Farmstar. Une telle approche entraînerait une consommation excessive de mémoire et dégraderait fortement les temps de réponse.

Le problème était donc le suivant :

- le modèle doit disposer des informations nécessaires pour résoudre une tâche ;
- il ne doit pas recevoir l'ensemble du dépôt ;
- les informations transmises doivent être pertinentes pour la requête ;
- le système doit pouvoir récupérer de nouvelles informations au cours de son exécution.

Cette contrainte a conduit à dissocier le raisonnement du modèle de la construction de son contexte.

4.1.2. Cas d'usage étudiés

L'intégration de l'intelligence artificielle dans le projet Farmstar a été envisagée autour de plusieurs activités susceptibles d'assister les développeurs :

- assistance au codage en temps réel ;
- analyse de tickets Jira ;
- revue automatisée de code ;
- génération de tests unitaires ;

- rétro-engineering de modules existants.

Ces usages ont en commun de nécessiter un accès à des informations propres au projet. Un modèle généraliste peut connaître Java, Spring Boot, JUnit ou Mockito, mais il ne connaît pas les conventions spécifiques de Farmstar, la structure du monorepo ou le contenu d'une classe particulière.

L'enjeu du projet n'était donc pas seulement de produire du texte ou du code, mais de construire un système permettant au modèle d'accéder de manière contrôlée aux informations nécessaires à la résolution de la tâche.

4.2. Le concept de Context Engineering

4.2.1. Limites de l'injection brute du contexte

La première approche envisageable consiste à fournir directement au modèle les fichiers nécessaires à la tâche. Cette solution devient rapidement impraticable lorsque le nombre de fichiers augmente.

Une tâche portant sur une classe Java peut par exemple nécessiter :

- la classe elle-même ;
- les interfaces qu'elle implémente ;
- les services qu'elle utilise ;
- les objets métier manipulés ;
- les tests déjà présents ;
- certaines conventions du projet.

Une simple recherche par mots-clés peut donc produire trop de résultats. À l'inverse, une recherche trop restrictive peut supprimer une information importante.

Il existe ainsi un compromis permanent entre la quantité d'informations fournies au modèle et leur pertinence. Cette problématique a été abordée dans le projet sous l'angle du **Context Engineering**.

L'objectif n'est plus seulement de rechercher des documents, mais de construire dynamiquement un contexte adapté à la tâche en cours.

4.2.2. Découpage sémantique du code

Une première étape consiste à ne pas découper le code uniquement selon une taille arbitraire en caractères ou en tokens.

Des parseurs syntaxiques tels que Tree-sitter peuvent être utilisés afin d'identifier des unités logiques du code, par exemple des classes ou des méthodes Java ou TypeScript.

Ce découpage permet de conserver une cohérence syntaxique. Un fragment correspond ainsi davantage à une unité compréhensible par le modèle qu'à une portion arbitraire d'un fichier.

Cette approche est particulièrement importante dans le contexte de l'analyse de code source. Un découpage effectué au milieu d'une méthode ou entre deux éléments syntaxiquement liés peut supprimer une partie du sens nécessaire au raisonnement.

4.2.3. Enrichissement par les métadonnées

Les fragments récupérés sont également associés à des informations permettant de conserver leur contexte d'origine.

Par exemple, un fragment peut être associé à :

- son chemin de fichier ;
- son module ;
- sa branche ;
- son emplacement dans le projet.

Le modèle ne reçoit donc pas uniquement le contenu du fragment. Il reçoit également des informations permettant de comprendre où celui-ci se situe dans l'architecture globale.

Cette distinction est importante dans un monorepo, où plusieurs modules peuvent contenir des classes ayant des noms similaires.

4.2.4. Recherche hybride

La recherche utilisée ne repose pas uniquement sur la similarité vectorielle. Deux types de recherche complémentaires sont utilisés.

La recherche dense par embeddings permet de retrouver des éléments présentant une proximité sémantique avec la requête. Elle est particulièrement utile lorsque la demande est formulée de manière conceptuelle.

À l'inverse, une recherche lexicale de type BM25 est pertinente lorsqu'un identifiant exact doit être retrouvé. Dans du code source, les noms de classes, méthodes ou variables constituent souvent des éléments déterminants.

La combinaison de ces deux approches permet donc de limiter une faiblesse importante des systèmes de recherche reposant uniquement sur les embeddings : une similarité sémantique élevée ne signifie pas nécessairement que le fragment contient l'identifiant exact recherché.

4.2.5. Re-ranking et skeletonization

Une étape de re-ranking permet ensuite de réévaluer la pertinence des premiers résultats obtenus.

Le principe est de ne pas transmettre au modèle l'ensemble des résultats trouvés, mais de sélectionner les fragments les plus pertinents.

Une autre stratégie étudiée consiste à utiliser une représentation **skeletonisée** des dépendances. Le code complet de la classe directement analysée peut être conservé tandis que le corps des méthodes de certaines classes dépendantes est masqué, en conservant leurs signatures.

Cette représentation permet de fournir au modèle une vision de l'architecture sans augmenter inutilement la taille du contexte.

L'ensemble de ces mécanismes constitue donc une chaîne de préparation du contexte. Le modèle de langage ne travaille pas directement sur le monorepo : il travaille sur une représentation sélectionnée et organisée des informations pertinentes.

4.3. Le Model Context Protocol : un catalogue d'outils métier

4.3.1. Du modèle de langage à l'agent

La construction du contexte ne suffit cependant pas à résoudre toutes les tâches. Dans certaines situations, le système doit pouvoir effectuer une action pendant son raisonnement.

Par exemple, lorsqu'un développeur demande d'analyser un fichier précis, l'agent doit pouvoir :

- identifier le fichier ;
- parcourir l'arborescence ;
- lire son contenu ;
- rechercher des éléments similaires ;
- consulter certaines connaissances ou conventions du projet.

Le Model Context Protocol (MCP) a été utilisé pour répondre à cette problématique.

L'idée est de fournir au modèle un ensemble d'outils clairement définis qu'il peut invoquer lorsque la tâche le nécessite. Le modèle n'accède donc pas directement au système de fichiers : il passe par une interface d'outils contrôlée.

4.3.2. Conception du serveur FastMCP

Un serveur FastMCP a été développé pour exposer les principales opérations nécessaires à l'exploration du projet.

Les outils développés couvrent plusieurs catégories :

- navigation dans la structure du projet avec `list_project_structure` et `list_project_files` ;
- lecture et synthèse du code avec `read_project_file` et `get_file_summary` ;
- recherche sémantique et recherche exacte avec `search_rag_code` et `search_by_regex` ;
- accès aux connaissances spécifiques du projet avec `list_and_read_skill`.

Cette organisation permet de distinguer clairement les capacités de l'agent de son raisonnement.

Le LLM décide qu'une information lui est nécessaire, mais l'accès effectif à cette information est réalisé par un outil contrôlé.

4.3.3. Complémentarité entre RAG et MCP

Le RAG et le MCP répondent ainsi à deux besoins différents.

Le RAG permet de retrouver les informations pertinentes à partir d'une base indexée. Il constitue principalement un mécanisme de récupération de contexte.

Le MCP fournit quant à lui une interface permettant à l'agent d'effectuer des opérations explicites sur son environnement.

Cette distinction devient importante dans l'architecture finale. Le modèle peut utiliser le contexte fourni par le système de recherche, mais peut également demander une nouvelle information via un outil MCP lorsqu'il constate que son contexte est insuffisant.

L'objectif est donc de construire un agent qui ne soit ni limité à un contexte statique, ni autorisé à accéder directement et sans contrôle à l'ensemble du système.

4.4. Expérimentation sous Open WebUI et motivation de la migration

4.4.1. Une première validation fonctionnelle

Avant de développer un orchestrateur spécifique, une première expérimentation a été réalisée avec Open WebUI.

Le prototype comportait trois agents principaux :

- un agent Farmstar MCP ;
- un agent Jira MCP ;
- un agent Ticket Solver.

Cette première architecture avait un objectif essentiellement exploratoire : vérifier la pertinence du concept et valider que les modèles pouvaient effectivement utiliser les outils MCP.

Cette étape a permis de confirmer la faisabilité technique de l'approche. Cependant, elle a également mis en évidence une limite importante qui ne concernait pas les capacités individuelles des agents, mais leur coordination.

4.4.2. Le problème de l'isolation des agents

Chaque agent disposait de son propre contexte de conversation. Les agents étaient donc capables de réaliser individuellement leurs tâches, mais ils ne formaient pas réellement un système collaboratif.

Cette différence est importante.

Considérons par exemple une demande telle que :

« Analyser le ticket FS-402 et modifier le service Java impacté. »

Cette tâche combine plusieurs opérations :

1. comprendre le ticket ;
2. identifier le module concerné ;
3. rechercher le code correspondant ;
4. analyser son fonctionnement ;
5. éventuellement identifier les tests existants ;
6. proposer ou générer une modification.

Avec une architecture composée de chatbots isolés, l'utilisateur doit assurer lui-même le passage d'une étape à l'autre. Les informations produites par un agent doivent être copiées vers le suivant.

Cette organisation présente deux problèmes.

Le premier est opérationnel : elle demande une intervention humaine importante.

Le second est qualitatif : lors du transfert manuel, une partie du contexte peut être oubliée ou reformulée. Le résultat produit par le second agent dépend alors de la manière dont l'utilisateur a retranscrit le travail du premier.

L'expérimentation Open WebUI a donc validé le concept d'agent individuel mais a montré que cette solution ne répondait pas complètement au besoin d'orchestration.

4.4.3. Reformulation du problème

La question n'était donc plus :

« Comment faire fonctionner plusieurs agents ? »

mais plutôt :

« Comment coordonner plusieurs agents spécialisés autour d'un état partagé, tout en permettant au système de choisir dynamiquement quelles étapes sont nécessaires ? »

Cette reformulation a conduit à abandonner le rôle d'Open WebUI comme couche principale d'orchestration au profit d'un middleware dédié.

Le choix s'est porté sur LangGraph.

4.5. Problématique d'orchestration

La migration vers LangGraph répond à plusieurs besoins identifiés lors de l'expérimentation précédente.

Premièrement, les informations produites par les différents agents doivent être conservées dans un état commun.

Deuxièmement, l'ordre d'exécution ne doit pas nécessairement être fixe. Une requête fournissant déjà le code à analyser ne nécessite pas forcément une nouvelle phase de collecte.

Troisièmement, certaines tâches peuvent être terminées plus tôt. Une simple demande d'analyse n'a par exemple pas nécessairement besoin de déclencher une génération de tests.

Enfin, les différentes capacités doivent pouvoir être regroupées dans des unités fonctionnelles indépendantes afin de faciliter l'évolution du système.

L'architecture devait donc répondre simultanément à quatre exigences :

- partager l'état entre les différentes étapes ;
- contrôler explicitement les transitions ;
- permettre des branches conditionnelles ;
- conserver une structure suffisamment modulaire pour ajouter de nouveaux agents et de nouveaux cas d'usage.

C'est précisément dans cette logique que le graphe d'agents a été conçu.

4.6. Conception de l'orchestrateur avec LangGraph

4.6.1. Principe du graphe d'état

LangGraph a été utilisé comme couche d'orchestration.

L'idée centrale est de représenter le workflow sous la forme d'un graphe composé de nœuds et de transitions.

Chaque nœud représente une étape de traitement. Il peut correspondre à un agent, à une opération de préparation ou à une fonction de routage.

Les transitions définissent quant à elles les étapes pouvant être exécutées ensuite.

L'intérêt de cette représentation est de rendre explicite le déroulement de la tâche. Contrairement à un simple enchaînement de fonctions, le système peut prendre des décisions intermédiaires et revenir vers certaines étapes lorsque le contexte le nécessite.

Le graphe constitue donc une représentation de l'organisation du raisonnement plutôt qu'une simple succession d'appels à un modèle de langage.

4.6.2. L'état partagé

L'élément central de l'orchestration est l'état partagé.

Chaque étape du graphe peut lire les informations nécessaires à son traitement et produire de nouvelles informations destinées aux étapes suivantes.

Dans le cas du workflow de couverture de tests, l'état peut notamment représenter :

- la demande initiale de l'utilisateur ;
- les informations identifiées lors de la collecte ;
- le code source pertinent ;
- les tests déjà présents ;
- les informations relatives aux écarts de couverture ;
- le plan de tests ;
- les résultats intermédiaires produits par les agents ;
- les éléments nécessaires à la génération finale.
- autres informations utiles à la tâche.

Cette organisation évite le transfert manuel du contexte entre les agents.

Un agent ne doit plus transmettre lui-même l'ensemble de son historique au suivant. Il met à jour l'état du graphe, et le nœud suivant récupère les informations dont il a besoin.

Cette séparation améliore également la lisibilité de l'architecture : les données partagées sont définies au niveau du workflow et non implicitement dans les prompts.

4.6.3. Le superviseur central

Le graphe principal est piloté par un agent Superviseur.

Celui-ci constitue le point d'entrée du système. Son rôle n'est pas de résoudre directement la tâche métier, mais de déterminer quelle capacité doit être mobilisée.

La requête utilisateur est donc d'abord analysée afin d'identifier son intention.

La sortie du superviseur est structurée avec Pydantic. Le système ne dépend ainsi pas uniquement d'une réponse textuelle telle que :

« Je pense qu'il faudrait utiliser le module de couverture. »

Il attend une structure correspondant à un ensemble de valeurs autorisées par l'architecture.

Cette contrainte permet de transformer une décision produite par le LLM en information exploitable par le programme.

Le superviseur joue ainsi un rôle comparable à celui d'un contrôleur dans une architecture logicielle classique : il ne réalise pas toutes les opérations, mais détermine quelle partie du système doit être appelée.

4.6.4. Routage dynamique

Une caractéristique importante de l'architecture est que le chemin d'exécution peut dépendre du contenu de la requête.

Le système analyse notamment si des informations importantes sont déjà présentes dans le prompt.

Par exemple, lorsqu'un utilisateur fournit directement du code ou des informations relatives aux écarts de couverture, le système peut éviter une phase de collecte qui serait redondante.

Ce mécanisme est appelé dans l'implémentation un routage court-circuit.

Il répond à un principe simple : une architecture agentique ne doit pas exécuter systématiquement toutes ses capacités lorsque certaines étapes sont inutiles.

Cette optimisation présente deux intérêts.

Le premier est la réduction du nombre d'appels au modèle de langage.

Le second concerne la pertinence du résultat. Chaque étape supplémentaire introduit potentiellement du bruit ou une transformation intermédiaire de l'information. Éviter une étape inutile permet donc également de conserver plus directement le contexte fourni par l'utilisateur.

4.6.5. Sous-graphes

Afin de conserver une architecture modulaire, les workflows métier sont organisés sous forme de sous-graphes.

Le graphe principal reste responsable de l'orientation générale de la requête, tandis qu'un sous-graphe prend en charge un cas d'usage particulier.

Cette organisation permet de séparer deux niveaux de raisonnement :

- le raisonnement global : **quelle capacité faut-il utiliser ?** ;
- le raisonnement métier : **comment réaliser cette capacité ?**.

Cette séparation est particulièrement utile lorsque plusieurs cas d'usage doivent coexister.

L'ajout d'un nouveau workflow ne nécessite alors pas de transformer l'intégralité du graphe principal. Il peut être développé comme une nouvelle unité fonctionnelle et raccordé au mécanisme de routage du superviseur.

4.7. Mise en œuvre du workflow TEST_COVERAGE

Le premier cas d'usage structurant de l'orchestrateur concerne l'analyse de la couverture des tests.

Ce choix est particulièrement intéressant pour valider l'architecture multi-agents car la tâche nécessite plusieurs étapes complémentaires. Elle combine en effet l'exploration du code, l'analyse des tests existants et la génération de nouveaux tests.

Le workflow a donc été décomposé en trois rôles :

- Gatherer ;
- Analyser ;
- Solver.

Cette décomposition permet de distinguer la collecte de l'information, son analyse et la production du résultat.

4.7.1. Les types de tâches :

Afin de mieux partitionner le workflow, une requête utilisateur est toujours associée à un type de tâche. Quelques exemples de types de tâches sont :

- EXPLAIN: l'utilisateur demande une explication sur un élément du code de test d'une feature.
- ANALYZE: l'utilisateur demande une analyse de la couverture des tests pour un élément du code.
- GENERATE: l'utilisateur demande la génération de tests pour un élément du code.
- AUDIT: l'utilisateur demande un audit complet de la couverture des tests pour un élément du code.

Ainsi, en fonction de la tâche détectée l'exécution du graphe diffère. Par exemple, si la tâche est de type EXPLAIN, le graphe ne passera par l'Agent Solver, mais uniquement par l'Agent Gatherer pour récupérer le contexte nécessaire et par l'Agent Analyser pour analyser le contexte et produire une explication. (Voir ci-dessous le comportement des Agents)

4.7.2. Le Gatherer : constitution du contexte technique

Le Gatherer constitue le point d'entrée métier du sous-graphe TEST_COVERAGE.

Son rôle est de rassembler les informations nécessaires à l'analyse.

C'est via cet Agent que le RAG est majoritairement utilisé.

Il identifie notamment le code source concerné et les tests déjà existants. Pour cela, il utilise les outils exposés par le serveur MCP.

La recherche peut donc combiner la navigation dans l'arborescence, la lecture de fichiers et la recherche ciblée dans le code.

Cette étape est importante car le modèle de langage ne doit pas être obligé de deviner quelles informations sont pertinentes. Le Gatherer transforme la demande générale en un contexte exploitable par les étapes suivantes.

Le résultat de cette phase alimente ensuite l'état partagé du graphe.

Il peut notamment contenir :

- les fichiers identifiés comme pertinents ;
- le contenu du code à analyser ;
- les tests déjà présents ;
- les informations permettant de comprendre le périmètre fonctionnel.

Le Gatherer réalise donc principalement un travail de récupération d'informations et non de génération.

4.7.3. L'Analyzer : identification des écarts

Le deuxième rôle est assuré par l'Analyzer.

À partir du contexte produit par le Gatherer, il cherche à expliquer, auditer ou encore analyser la couverture des tests.

Cette étape constitue la transition entre la collecte et la génération dans le cas d'une requête de type GENERATE.

L'Analyzer ne doit pas simplement demander au modèle de produire immédiatement du code. Il établit d'abord un plan de test.

Cette séparation permet de rendre le raisonnement plus explicite :

«

1. Qu'est-ce qui existe ?
2. Qu'est-ce qui n'est pas suffisamment couvert ?

3. Quels scénarios doivent être testés ?
4. Quels tests sont nécessaires pour couvrir ces scénarios ?
5. Seulement ensuite : comment générer ces tests ?

»

L'utilisation d'une étape d'analyse intermédiaire permet ainsi de réduire le risque de produire directement des tests sans avoir correctement identifié le besoin.

4.7.4. Le Solver : génération des tests

Le Solver intervient après l'analyse.

Son rôle est de transformer le plan de test en suites de tests unitaires concrètes.

Dans le périmètre étudié, les technologies ciblées sont notamment JUnit 5 et Mockito.

Le Solver dispose donc des informations collectées par le Gatherer et du plan établi par l'Analyzer.

Cette organisation évite de demander au générateur de tests de reconstruire lui-même l'ensemble du contexte.

La génération devient alors une étape spécialisée dans un workflow plus large.

4.7.5. Pourquoi trois agents ?

La séparation Gatherer / Analyzer / Solver n'a pas été réalisée uniquement pour multiplier le nombre d'agents.

Elle répond à une séparation fonctionnelle.

Le Gatherer répond à la question :

« Quelles informations sont disponibles ? »

L'Analyzer répond à :

« Que faut-il faire à partir de ces informations ? »

Le Solver répond enfin à :

« Comment produire concrètement le résultat demandé ? »

Cette séparation rend les responsabilités plus faciles à comprendre et permet également d'améliorer indépendamment chaque étape.

Elle permet notamment de faire évoluer la méthode de recherche sans modifier la génération des tests, ou d'améliorer l'analyse sans modifier les outils MCP.

4.8. Exécution conditionnelle et optimisation du workflow

L'un des enseignements de l'expérimentation a été que la présence de plusieurs agents ne signifie pas que tous doivent être exécutés à chaque requête.

Un workflow strictement linéaire du type :

« Gatherer -> Analyser -> Solver »

est simple à comprendre mais devient inefficace lorsqu'une partie des informations est déjà disponible.

Une requête peut par exemple fournir directement le code à analyser. Dans ce cas, exécuter une nouvelle collecte complète est inutile.

De même, certaines requêtes peuvent demander uniquement une analyse de la couverture. Dans ce cas, le Solver n'a pas besoin d'être appelé.

L'architecture a donc été affinée afin de supporter des requêtes partielles.

Le sous-graphe peut ainsi adapter son exécution au contexte disponible.

Cette évolution est importante du point de vue de l'ingénierie. Elle montre que l'objectif du projet n'est pas de construire un pipeline fixe, mais un système capable d'adapter son parcours à la tâche.

L'exécution conditionnelle permet également de limiter les appels au LLM. Ce point est particulièrement important avec des modèles locaux, pour lesquels les temps d'inférence peuvent devenir significatifs lorsque plusieurs étapes de raisonnement sont enchaînées.

4.9. Routage et boucles de rétroaction

Le graphe ne se limite pas à une succession de nœuds.

L'intérêt de l'approche réside également dans la possibilité de représenter des transitions conditionnelles et des boucles.

Dans un pipeline traditionnel, une erreur ou une information manquante doit souvent être traitée explicitement dans chaque fonction.

Dans une architecture orientée graphe, la logique de contrôle peut être représentée directement dans les transitions.

Par exemple, si l'étape d'analyse détecte que le contexte fourni par la collecte est insuffisant, le workflow peut être conçu pour retourner vers une étape de récupération d'informations plutôt que de poursuivre directement vers la génération.

Cette possibilité est particulièrement intéressante pour un système agentique. Le raisonnement peut produire une nouvelle information sur les besoins du workflow.

L'agent n'est donc plus seulement un composant de calcul placé dans une pipeline fixe : il participe indirectement au choix du chemin d'exécution.

Cette logique a également conduit à envisager un routage interne plus autonome des sous-graphes, capable de supprimer certaines étapes lorsqu'elles ne sont pas pertinentes.

4.10. Architecture orientée objet et robustesse des agents

La mise en place de LangGraph ne constituait pas à elle seule une architecture suffisamment maintenable.

Un premier risque identifié était de multiplier les classes d'agents contenant chacune une logique différente de chargement des prompts, de configuration et d'accès aux outils.

Pour éviter cette duplication, le framework d'agents a été refactorisé autour d'une architecture orientée objet.

4.10.1. Classe abstraite Agent

Une classe abstraite Agent sert de base commune aux différents agents.

Elle encapsule notamment :

- le cycle de vie d'un agent ;
- le chargement des configurations ;
- les prompts ;
- les compétences ou **skills** ;
- l'accès au registre des outils MCP.

Les agents spécialisés peuvent ainsi se concentrer sur leur comportement métier plutôt que de réimplémenter les mécanismes communs.

Cette approche rapproche l'architecture du graphe d'une architecture logicielle classique : chaque agent possède une responsabilité identifiée et repose sur un contrat commun.

Cela permet une prise en main plus rapide pour un développeur qui souhaiterait ajouter un nouveau cas d'usage.

4.10.2. Séparation des configurations

Les prompts et les **skills** ont été séparés du code de l'agent.

Cette séparation présente plusieurs avantages.

Elle permet d'abord de modifier le comportement attendu d'un agent sans modifier sa logique d'exécution.

Elle facilite ensuite les expérimentations. Dans un projet R&D, les prompts sont susceptibles d'évoluer fréquemment. Les maintenir dans des fichiers dédiés évite donc de mélanger les réglages expérimentaux avec le code structurel du framework.

Enfin, cette organisation rend plus simple la comparaison de plusieurs configurations.

4.10.3. Typage et registres

Le routage repose également sur des types explicites.

Des énumérations telles que `SubGraphType`, `SubAgentType` et `AgentTool` permettent d'éviter l'utilisation dispersée de chaînes de caractères.

Cette décision peut sembler secondaire, mais elle devient importante lorsque le nombre d'agents augmente.

Un routage fondé sur des chaînes libres peut conduire à des erreurs difficiles à détecter :

« test_coverage » « TEST_COVERAGE » « test-coverage » »

représenteraient potentiellement la même intention mais constitueraient trois valeurs différentes.

L'utilisation de types explicites réduit ce risque et centralise les valeurs autorisées.

4.11. Sorties structurées et validation des décisions du LLM

Un autre problème rencontré dans une architecture agentique concerne la fiabilité des sorties du modèle.

Un LLM produit naturellement du texte. Or le programme d'orchestration attend des informations structurées.

Le superviseur doit par exemple retourner une décision de routage exploitable par le graphe.

Il n'est donc pas suffisant de demander au modèle de répondre sous la forme d'un JSON dans son prompt. Le système doit également vérifier que la réponse respecte réellement le schéma attendu.

4.11.1. Pydantic

Pydantic est utilisé pour définir les structures attendues.

Le modèle doit ainsi produire des données correspondant à des types précis.

Cette approche permet de rapprocher le comportement probabiliste du LLM des contraintes déterministes du programme.

Le modèle reste responsable de la décision, mais la structure de cette décision est contrôlée par l'application.

4.11.2. Instructor et mécanisme de retry

La bibliothèque Instructor a également été étudiée afin de renforcer cette gestion des sorties structurées.

L'intérêt recherché est notamment de pouvoir gérer les erreurs de formatage ou les champs manquants avec une logique de correction automatique.

Dans le contexte de l'orchestration, ce mécanisme est particulièrement utile car une sortie invalide du superviseur peut empêcher le graphe de poursuivre son exécution.

La validation structurée constitue donc une forme de frontière entre la partie probabiliste du système et sa partie logicielle déterministe.

4.12. Déploiement et industrialisation du prototype

4.12.1. Conteneurisation

Une fois les différents composants assemblés, l'architecture devait pouvoir être exécutée de manière reproductible.

L'ensemble du prototype a donc été conteneurisé avec Docker.

Docker Compose permet d'orchestrer les différents services nécessaires :

- l'API d'ingestion ;
- le serveur FastMCP ;
- le moteur LangGraph ;
- l'interface Streamlit ;
- l'API et moteur LangFuse : qui fournit les métriques et la traçabilité.

Les composants peuvent ainsi être lancés dans un environnement isolé et communiquer sur un réseau interne.

Cette organisation réduit la dépendance à l'environnement de développement local, aux APIs externes (LangSmith) et facilite la reproduction du PoC.

Elle constitue également une première étape vers une éventuelle intégration plus large dans l'environnement de développement de l'équipe.

4.12.2. Interface d'ingestion et administration

Une interface Streamlit complète le système afin de faciliter certaines opérations d'administration et d'ingestion.

Cette interface ne constitue pas le cœur du raisonnement agentique. Elle sert principalement de couche d'interaction avec les composants du prototype.

La séparation entre l'interface, le moteur d'orchestration et les outils MCP permet ainsi de conserver une architecture modulaire.

Le remplacement de l'interface utilisateur ne nécessite pas de modifier le fonctionnement interne du graphe.

5. Observabilité et traçabilité

Un système multi-agents est difficile à déboguer si l'on ne peut pas observer les étapes intermédiaires de son exécution, ainsi que sa réflexion (CoT).

Une requête peut en effet provoquer plusieurs appels au LLM, plusieurs appels MCP et plusieurs décisions de routage, voir même boucler sur certaines étapes.

Lorsqu'un résultat est incorrect, il est donc nécessaire de savoir quelle étape a produit l'information problématique.

LangFuse a été intégré au prototype afin d'apporter cette observabilité.

L'objectif est notamment de pouvoir suivre :

- les différentes étapes d'exécution ;
- les appels aux outils ;
- la latence ;
- l'utilisation des tokens ;
- les interactions entre les composants.
- posséder une solution Open Source pour la traçabilité des requêtes et des résultats.

Cette instrumentation est particulièrement importante dans une démarche R&D. Elle permet de distinguer une erreur provenant du modèle, du contexte, du routage ou d'un outil.

Elle fournit également les éléments nécessaires à de futurs benchmarks.

Il existait bien une solution « clefs en main » (LangSmith créé par l'équipe de développeurs de LangGraph) mais elle n'était pas Open Source et demandait une connexion à un compte externe type Github, Google, etc.

Dans une architecture agentique, la qualité d'un système ne peut donc pas être évaluée uniquement à partir de sa réponse finale. Il faut également pouvoir analyser le chemin qui a conduit à cette réponse.

5.1. Déroulement d'une requête complète

Afin de mieux comprendre l'architecture, il est possible de suivre le traitement d'une requête de couverture de tests.

Supposons qu'un développeur demande au système d'améliorer les tests d'un service Java.

- 1) La première étape consiste à transmettre cette requête au superviseur.
- 2) Le superviseur analyse l'intention et identifie le workflow TEST_COVERAGE.
- 3) Le graphe transfère alors l'exécution vers le sous-graphe correspondant.

- 4) Le Gatherer examine le contexte disponible. Si le développeur n'a fourni aucun code pertinent, il utilise les outils MCP afin d'identifier le service concerné, retrouver son fichier source et rechercher les tests existants.
- 5) Les informations obtenues sont ensuite ajoutées à l'état partagé.
- 6) L'Analyzer récupère cet état et analyse le code ainsi que les tests existants. Il identifie les scénarios insuffisamment couverts et construit un plan de test.
- 7) Ce plan est à son tour ajouté à l'état du workflow.
- 8) Le Solver peut alors utiliser les informations précédentes pour générer les tests correspondants en JUnit 5 et Mockito.
- 10) Le résultat final est produit à partir de l'ensemble du contexte construit progressivement.

Aucun agent n'a besoin de posséder individuellement l'intégralité de la tâche et on évite ainsi les problèmes de transfert manuel d'informations, d'hallucinations ou de perte de contexte.

Chaque agent réalise une partie spécialisée du travail et communique son résultat par l'intermédiaire de l'état du graphe.

5.2. Bilan de l'architecture

La migration d'Open WebUI vers LangGraph n'a donc pas constitué un simple changement de bibliothèque.

Elle correspond à une évolution du modèle d'architecture.

Dans la première approche, plusieurs agents spécialisés étaient juxtaposés. La coordination dépendait en grande partie de l'utilisateur.

Dans la seconde approche, les agents deviennent des composants d'un système orchestré.

Le superviseur détermine le workflow approprié, l'état partagé conserve les résultats intermédiaires, les sous-graphes organisent les capacités métier et les transitions conditionnelles permettent d'adapter l'exécution au contexte.

Cette évolution apporte plusieurs bénéfices.

Premièrement, elle réduit la dépendance aux manipulations manuelles entre agents.

Deuxièmement, elle rend explicite le chemin d'exécution.

Troisièmement, elle facilite l'ajout de nouveaux workflows.

Enfin, elle permet d'introduire progressivement des mécanismes plus avancés de routage et de contrôle.

5.3. Limites identifiées

Malgré ces résultats, le système développé reste un prototype de R&D et ne doit pas être présenté comme un système complètement industrialisé.

La première limite concerne la dépendance aux performances du modèle local. La qualité du routage, de l'analyse et de la génération reste liée aux capacités du modèle utilisé.

La seconde concerne le coût d'un workflow multi-agents. Chaque étape supplémentaire peut nécessiter une nouvelle inférence et donc augmenter la latence globale.

Cette contrainte justifie les travaux réalisés sur les routages court-circuits et l'exécution conditionnelle.

Une troisième limite concerne la fiabilité du raisonnement. Même lorsqu'une sortie respecte correctement son schéma Pydantic, son contenu peut rester incorrect. La validation structurelle garantit donc la forme de la réponse, mais pas sa pertinence métier.

Enfin, les mécanismes de génération de code nécessitent toujours une validation par le développeur. Le rôle du système est d'assister le développeur et de réduire le travail de recherche et de préparation, et non de supprimer les mécanismes classiques de revue et de validation du code.

5.4. Résultats et perspectives

Le travail réalisé a permis d'obtenir un PoC multi-agents opérationnel basé sur LangGraph et MCP, capable d'analyser du code source et de construire des plans de tests.

La principale contribution du travail réside toutefois dans l'architecture mise en place.

Le prototype permet désormais de considérer l'IA comme un système composé de plusieurs capacités spécialisées plutôt que comme une succession de conversations indépendantes.

Cette architecture fournit également une base pour de futures extensions.

Un premier axe consiste à rendre le routage des sous-graphes davantage autonome afin que les étapes inutiles soient automatiquement évitées.

Un second axe concerne l'intégration de nouveaux outils métier. Les serveurs MCP pourraient notamment être étendus afin d'interagir directement avec Jira ou SonarQube.

Un troisième axe consiste à intégrer l'orchestrateur dans les chaînes de développement existantes. À terme, le système pourrait par exemple être déclenché automatiquement lors de certaines étapes d'une Merge Request afin d'assister la revue de code ou l'analyse de tests.

Ces évolutions devront cependant être accompagnées d'une évaluation plus systématique de la qualité des résultats, de la latence et du coût d'exécution.

L'enjeu de la suite du projet n'est donc plus uniquement de démontrer qu'un LLM peut produire une réponse pertinente. Il consiste à déterminer dans quelles conditions une architecture agentique peut être suffisamment fiable, observable et reproductible pour être intégrée dans les pratiques quotidiennes des développeurs.

6. Synthèse et Bilan

6.1. Bilan technique et résultats obtenus

L'ensemble des objectifs fixés au début du stage a été atteint, apportant des améliorations mesurables sur le projet Farmstar :

- **Intégration Continue (CI/CD)** : Temps d'exécution du build backend réduit de 70% sur GitLab CI (de 19:18 à 5:45 min), et build local accéléré de 17 min à 3 min grâce à un taux d'utilisation du cache Spring de 99,5%.
- **Tests Frontend** : Suite de smoke tests Playwright 3 fois plus rapide que Cypress (1 min 45 s vs 5 min 28 s).
- **Qualité du code** : Interception des régressions avant la CI via des hooks Git pre-commit et pre-push ciblés sur les fichiers modifiés.
- **R&D IA Agentique** : Réalisation d'un PoC multi-agents opérationnel basé sur LangGraph et MCP, capable d'analyser le code source et de générer des plans de tests.

6.2. Estimation des gains pour l'entreprise

- **Productivité Développeur** : L'accélération des builds et la réduction de la boucle de feedback font gagner en moyenne 30 à 45 minutes par jour et par développeur sur l'équipe Farmstar.
- **Réduction des coûts d'infrastructure** : La baisse de la durée des jobs GitLab CI réduit directement le temps de calcul des runners.
- **Pérennité du Code** : La standardisation des pratiques de tests et les guides rédigés sur le wiki garantissent une baisse durable de la dette technique.

6.3. Bilan personnel

Ce stage de fin d'études a été une expérience d'ingénieur complète. Sur le plan technique, il m'a permis d'approfondir mes connaissances des entrailles de Spring Boot, du scripting CI/CD avancé, et des architectures émergentes d'IA agentique (LangGraph, MCP). Sur le plan méthodologique, j'ai développé ma capacité à mener des audits de performance, à prendre des décisions d'architecture en autonomie, et à communiquer mes choix à travers des présentations techniques et de la documentation.

7. Conclusion

7.1. Synthèse globale

Ce projet de fin d'études au sein de l'agence Magellium s'inscrit dans une démarche globale d'optimisation de l'ingénierie logicielle. Qu'il s'agisse de résoudre les problématiques de performance de la CI/CD ou de concevoir l'architecture IA de demain, l'approche retenue a toujours privilégié la rigueur architecturale, la qualité de code et l'autonomie des développeurs.

7.2. Perspectives

Plusieurs axes d'évolution peuvent prolonger ces travaux :

1. **Industrialisation du PoC IA** : Connecter l'orchestrateur LangGraph directement aux pipelines GitLab de Magellium pour automatiser les revues de code et l'audit de sécurité sur chaque Merge Request
2. **Extension des Serveurs MCP** : Ajouter des intégrations directes vers les APIs Jira et SonarQube pour permettre à l'agent de résoudre des tickets de bout en bout.
3. **Généralisation du pattern Hydrateur** : Étendre le GenericHydrator aux autres microservices du projet Farmstar.

8. Annexes

8.1. Annexe 1 : Tableau d'Analyse Réflexive des Compétences (Obligatoire UTBM)

Domaine de Compétence	Compétences Mobilisées / Développées	Projets / Actions Concrètes
Ingénierie Logicielle & Architecture	Conception orientée objet, design patterns (Factory, Hydrator, Observer), refactoring de code legacy Java/Spring Boot.	Développement du GenericHydrator, unification des contextes de test Spring (AbstractIntegrationTest).
DevOps & Qualité Logicielle	Optimisation de chaînes CI/CD, métrologie/ profiling, écriture de hooks Git (pre-commit/ pre-push), conteneurisation Docker.	Réduction du temps de build GitLab CI de 70%, benchmark Playwright vs Cypress, dockerisation du PoC IA.
Intelligence Artificielle & R&D	Orchestration d'agents LLM, protocole MCP, RAG (Vector DB Qdrant), typage strict des sorties (Instructor/Pydantic).	Conception de l'architecture multi-agents LangGraph et du serveur FastMCP pour le projet Farmstar.
Gestion de Projet & Posture	Autonomie, démarche « Fast-Fail », animation de présentations techniques, rédaction de documentation et wiki.	Animation de la restitution technique (28 slides) et rédaction des guides de bonnes pratiques de test.

8.2. Annexe 2 : Bibliographie & Sources Techniques (Indispensable UTBM)

- **Spring Framework Documentation** – Spring Test Context Management & Caching, 2026.
- **LangChain & LangGraph Documentation** – Multi-Agent Orchestration & State Management, 2026.

- **Model Context Protocol (MCP) Specification** – Anthropic Open Standard for AI Tool Integration, 2025-2026.
- **Playwright Frontend Testing** – Network Interception and Mocking Strategies, Fast End-to-End Execution, 2026.
- **Instructor Python Library** – Structured Outputs for LLMs via Pydantic Validation, 2026.
- **Chiffres clefs** - <https://www.pappers.fr/entreprise/magellium-450550991>

8.3. Annexe 3 : Analyse Framework de test Frontend

Contexte et Objectifs

Dans le cadre de l’amélioration de la couverture des tests du project Farmstar, il a été décidé d’adopter un framework de tests E2E afin de compléter les tests d’intégration existants. Ce document présente une analyse comparative entre plusieurs frameworks populaires (Cypress, Playwright, Selenium) pour déterminer la solution la plus adaptée aux besoins du projet.

Analyse Comparative des Frameworks de Tests E2E

Les critères d’évaluation retenus sont les suivants :

- **Facilité d’Intégration** : Capacité à s’intégrer dans la pipeline CI/CD existante.
- **Performance** : Temps d’exécution des tests et consommation de ressources.
- **Facilité d’Écriture des Tests** : Simplicité de la syntaxe et des outils de développement.
- **Support et Communauté** : Disponibilité de ressources, documentation et support communautaire.

Framework	Facilité d’Intégration	Performance	Facilité d’Écriture	Support et Communauté
Cypress	Excellente (Plugins CI/CD disponibles)	Moyenne (exécution parallèle locale possible mais non recommandée)	Très simple (interface User Friendly, outils de développement intégrés)	Large et active (nombreux plugins et ressources)
Playwright	Bonne (Support CI/CD en développement)	Rapide (exécution parallèle)	Simple (outils de développement intégrés)	En croissance rapide (soutenu par Microsoft)

Selenium	Moyenne (Intégration possible mais plus complexe)	Moins rapide (exécution plus lente, moins d'optimisations modernes)	Verbeux (API explicite mais lourde)	Immense (Standard de l'industrie, mais en déclin)
----------	--	--	-------------------------------------	--

Métrique / Critère	Cypress (Existant)	Playwright (Cible)	Gain / Impact
Temps d'exécution global	5 min 28 s	1 min 45 s	Temps divisé par 3
Consommation CPU (User time)	1 min 38 s	8.5 s	Réduction de 91%
Taux de succès de la suite	88,8 % (flaky tests)	100 %	Stabilité maximale
Architecture d'exécution	Proxy réseau lourd	Protocole DevTools natif	Faible empreinte CI

10.1. Annexe 4 : Analyse outils de pré-commit

Analyse des Outils d'Analyse Statique Java (Pre-commit)

L'intégration de « Pre-commit hooks » permet de garantir que tout code commité respecte les standards de qualité avant même l'entrée dans la pipeline CI/CD. Voici une comparaison des outils standards de l'écosystème Java.

Outil	Type d'analyse	Adéquation Pre-commit	Performance	Complexité Config.
Checkstyle	Style et conventions (indentation, nommage)	Excellente (agit sur les sources, très léger)	Très rapide	Moyenne (Fichier XML standard)
PMD	Bonnes pratiques, Code mort, Complexité cyclomatique	Très bonne (agit sur les sources)	Rapide	Moyenne (Règles intégrées)
SpotBugs	Détection de bugs (NullPointer, ressources non fermées)	Moyenne (Nécessite la compilation des classes)	Moins rapide (Analyse du bytecode)	Élevée (Plugins Gradle/Maven)
SonarLint	Qualité globale et Sécurité (Hotspots)	Faible (Conçu pour IDE ou mode connecté)	Lente pour un hook	Élevée (Nécessite contexte projet)

12. Sources :